JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 3

Classes and Objects

Learning outcomes

Upon completing this topic you should be able to:

- Declare a class
- Differentiate between member variables and member functions
- Define member functions
- Create objects from a class
- Acess data members

3.1. Introduction to Classes

Classes are fundamental building blocks of object-oriented programs.

A class binds data and functions together.

The data and functions of a class are its members.

The data members and member functions details are controlled using private, public and protected keywords.

private, public and protected keywords are also referred to as access specifier and are always followed by a colon (:).

The private section of a class has data members and member functions that are available only within the class(except by friend functions).

The public section makes the data members and member functions within it to be available to all class members and to the other classes members and applications.

The protected section has data members and member functions that are available to the class itself and any other derived class.

The beginning of each section is marked by keywords private:, public: and protected: By default data members and member functions are private.

Normally the data members are put in the private section of a class and the member functions are put in the public section.

3.2. Declaring a class

Specifying the data members and member functions of a class is known as declaring a class.

The syntax for defining a class is:

```
class class_name
{
        private:
                data_type member1;
                data_type member2;
                . . .
                data_type membern
        public:
                method function1;
                method function2;
                . . .
};
Example:
class Rectangle
```

```
{
        private:
                int length;
                int width;
        public:
                void print_dimension(void);
};
```

The class definition begins with class keyword followed by the name of the class. The name of a class is an identifier and it is not supposed to be a reserved word. The body of a class is enclosed within the opening curly brace ({) and closing curly brace (}). The closing curly brace is terminated by a semi-colon (;). The body of a class consists of data members and member functions.

Back Close Doc Doc

3.3. Defining Member Functions

There are two ways of defining member functions. a) Inside the class definition b) outside the class definition

- Definition a member function outside a class definition

The members functions declared within a class can be defined outside a class.

The definitions of these functions are just the same as definition of the normal functions(functions that are not members of a class). But unlike normal functions member functions defined outside a class definition are qualified with the name of the class that they belong to. The name of the class and the member function are separated using the scope resolution operator(: :).

The syntax of defining a member function outside a class is:

return-type class-name :: function-name (parameter list) { function body }

For instance, consider the print_dimension function declared in the Rectangle class above. It may be defined as follows: void Rectangle :: print_dimension(void)

{

cout<<The Length is: "<<length <<"cm\n" <<"The width is : "<<width <<"cm\n" <<endl;

}

- Definition a member function inside a class definition

Member functions can be defined within a class definition. Still consider the find_area() function declared in the class Rectangle above.

This function can be defined in the class as shown below.

```
class Rectangle
{
private:
int length;
int width;
public:
void print_dimension(void)
{
cout<<”The Length is: “<<length <<”cm\n” <<”The width is : “<<width <<”cm\n”<<endl;
}
};
```

Note:

3.4. Creating objects from a class

Once a class has been defined, it can be instantiated.

Instantiating a class is creating an object from a class.

The objects that are created are the ones that are used to call member functions of the class.

A class that can be instantiated is referred to as a concrete class.

There are those classes that can not be instantiated.

Such classes have at least one member a pure virtual function. Such classes are referred to as abstract classes.

The syntax for instantiated a class is:

- Form 1

class-name variable-name;

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

e.g.

Rectangle rect1;

b) Form 2

class-name variable-name(arguments);

e.g.

Rectangle rect2(4, 5);

The above two statements creates a class variables rect1 and rect2 of type Rectangle. A class variable is an object. In that case rect1and rect2 are objects of type Rectangle.

3.5. Accessing class members

All private data members are accessible only by class member functions(this restriction does not apply to friend functions).

It is illegal to have a statement for example within the main method that accesses the value stored in data member.

External functions access private data members by calling public members of a class. The syntax for doing this is

object-name. function-name(arguments);

This is what is referred to as message passing. The receiver of the message is object-name. The name of the message is function-name. The information that is passed to the object in order for the object to respond to the message is “arguments”

E.g.

rect1.print_dimension();

Below is an example of a C++ program that is made of a class

/* ____

Rectangle.cpp

This program demonstrates the use classes in a

C++ program

———————- */

```
#include<iostream.h>
class Rectangle
{
private:
int length;
int width;
public:
Rectangle(int l, int w);
void print_dimension(void);
```

```
};
int main(void)
{ Rectangle rect1(5, 3);
rect1.print_dimension();
return 0;
}
/* ________________________________________________

Definition of Rectangle member function
____________________________________________________

____________- */
Rectangle :: Rectangle(int l, int w)
{
length=l;
```

```
width=w;
}
/*______________________________________________________
Definition of print_dimension member function
______________________________________________________
_____________- */
void Rectangle :: print_dimension(void)
{
cout<<"The Length is: "<<length <<"cm\n" <<"The width is : "<<width <<"cm\n" <<endl;
}
```

If you compile and execute this program, the output is

The Length is: 5cm The width is : 4cm

Revision Questions

EXERCISE 1. ✍ Differentiate Rectangle rect1; and Rectangle rect2()

Example ✏. Differentiate between function declaration and function definition

Solution: Function declaration is only function header while function definition has function header and function body □

EXERCISE 2. ✍ Differentiate data members and member functions

EXERCISE 3. ✍ Explain the use private, protected and public keywords

EXERCISE 4. ✍ Explain the difference between internal and

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

external definition of member functions

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

Solutions to Exercises

Exercise 1.

Rectangle rect1; This is a call to a default constructor in a class Rectangle

Rectangle rect2(); This is a declaration of a function named rect2() that is not passed any parameters. The function returns a value of type Rectangle.

Exercise 1